

Software Engineering

Multi-Process Computing

But First...Memory Management

Stack/Heap: Beyond the Basics

Data on the stack gets cleaned up when it goes out of **scope**

What about the heap?

C/C++: manual memory management, every **new** must have a **free**

→ Turns out this is **extremely** error prone (std::unique_ptr helps)

Java/Python/C#, etc: Track references to objects in the heap as a **reference counter**

Reference Counting

Not widely used - prone to memory leaks

- Tracks how many pointers point to a given object
- When the count reaches 0, garbage collect the object

Downsides:

- Need to instrument every pointer assignment
- Consider circular linked lists, or doubly linked lists

Mark And Sweep

Apply performance tuning technique: add **batching**

Two phases of garbage collection:

"Mark" - track all **reachable** objects and mark them

"Sweep" - remove all unmarked objects

"Mark" Phase

```
String s = new String("Hello World");  
s = new String("New Text");
```

Start at "root" objects:

- Anything pointed to by a pointer currently on the stack (technically, **a** stack)
- Any globally-accessible static state

Add these to the "set of current pointers"

Repeat:

- For everything pointed to by a current pointer, set the 'mark' bit to 1
- For any pointers contained in a pointed-to object, add that pointer to the next set of pointers
- Once we're done with this pass, update the "set of current pointers" to the "next set of pointers"

"Sweep" Phase

Go through all objects in the heap:

- If the 'mark' bit is not set, free the memory for that object

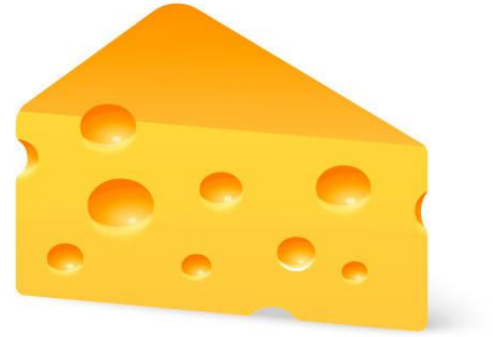
Avoid Fragmentation: Mark and Compact

Freeing lots of little bits of memory leaves
"holes" in the heap

- Makes allocating new memory expensive

Alternate version:

- Copy all objects with a mark bit of 1 somewhere else
- Free everything left behind



Demo!

Optimization: No need to sweep everything

Fun fact: objects tend to either be short-lived or live forever

⇒ Have multiple "spaces" in the heap, don't sweep all of them every time

Spaces for Java:

Young generation (Eden + survivor)

Old generation

Optimization: No need to sweep everything

When compacting, copy everything to a one-older generation

"Minor" gc - just Eden space

"Major" gc - progressively older spaces

Garbage Collection

Pros:

- Avoid (most) memory leaks
- Automated
- Runs in the background

Cons:

- Freezes the world (sort of, there have been improvements)
- Can take a long time
- Tuning this is tricky

Garbage Collection Tuning (Unusual)

`System.gc()` - suggests to the JVM that now would be a good time to run garbage collection

Many JVM parameters to tune garbage collection performance, which can be important for large long-running servers

`-Xmx`, `-Xms`, `-XX:ParallelGCThreads`, many more

This is also where **weak** and **soft** references come in:

Weak reference - do not increase the reference counter to the object (doesn't set the mark bit)

Soft reference - only GC this object as a last resort/"kind of" count the reference

Multi-Process Computing

Single servers have upper bounds on IO and CPU (fairly large these days, but still noticeable)

Threads require a shared heap, and therefore must run on one server

To continue to scale software, we need to move Beyond Threads

Splitting a job across multiple machines is called **sharding** (each machine is a **shard**)

Multi-Process Advantages

Arbitrarily scalable:

- disk space
- CPU
- RAM

Cost-effective scaling (similar to RAID for disks)

Multi-Process Oversimplifications

Not more secure!

- Security is a reason why an OS will have many processes

Multi-Process Oversimplifications

Fault tolerance: (effective) multi-process requires that you add fault tolerance, it does not get that for free

Why fault tolerance?

- Similar to RAID; as number of servers increases, time to single server failure decreases

Fault tolerance requirements:

- Failure detection
- Retry for failures
- Known state after failure: idempotent (retriable) operations, or checkpoints + rollback

Multi-Process Computing

Make fault tolerance easier by making operations **stateless**

- Relates to a 'design for debugability' concept: keep state as local as possible
- Refactor code to keep logic independent (top-down design helps here)
- Streaming algorithms (constant space, constant time, easily sharded)

Design For Multi-Process

Algorithm must be **shardable**

- Identify "batches" of work that can happen independently
- Send one batch to each shard

Overall result must be computable from results of each shard

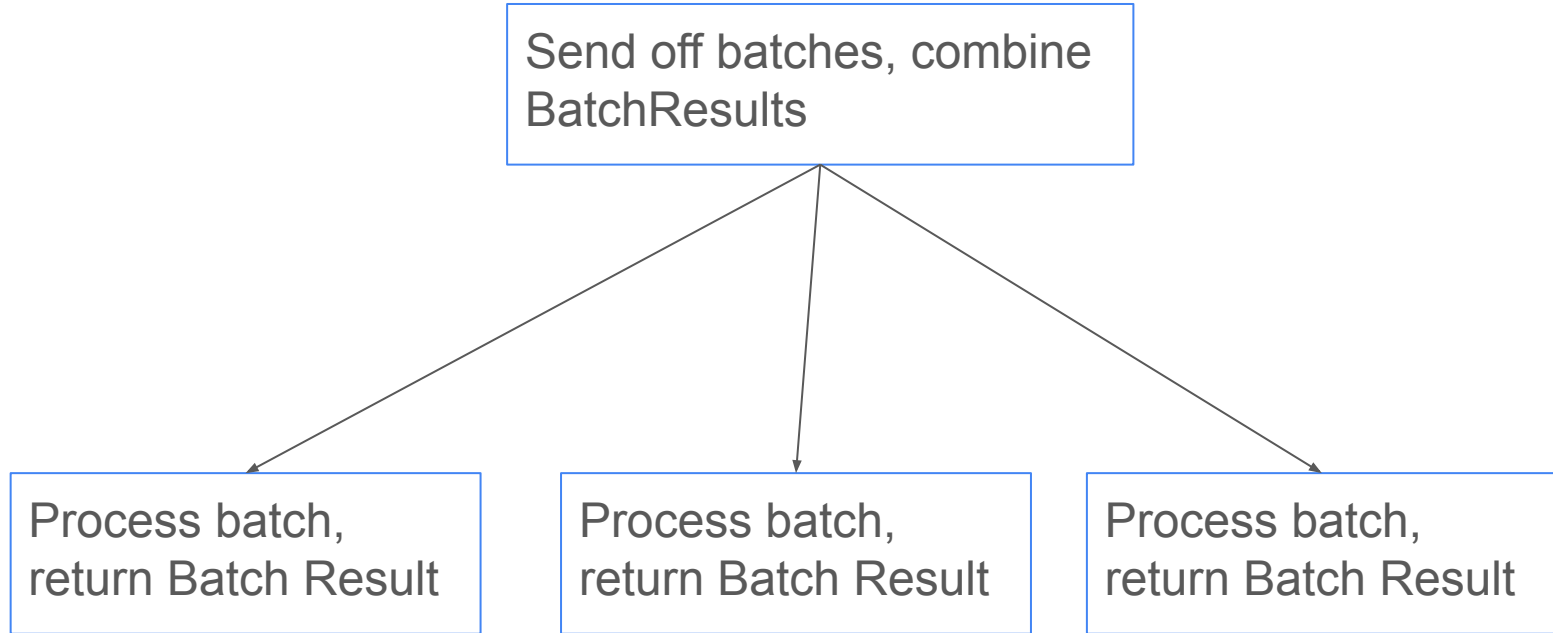
Your Turn!

Overall goal: build some multi-process 'find top 3 values' code

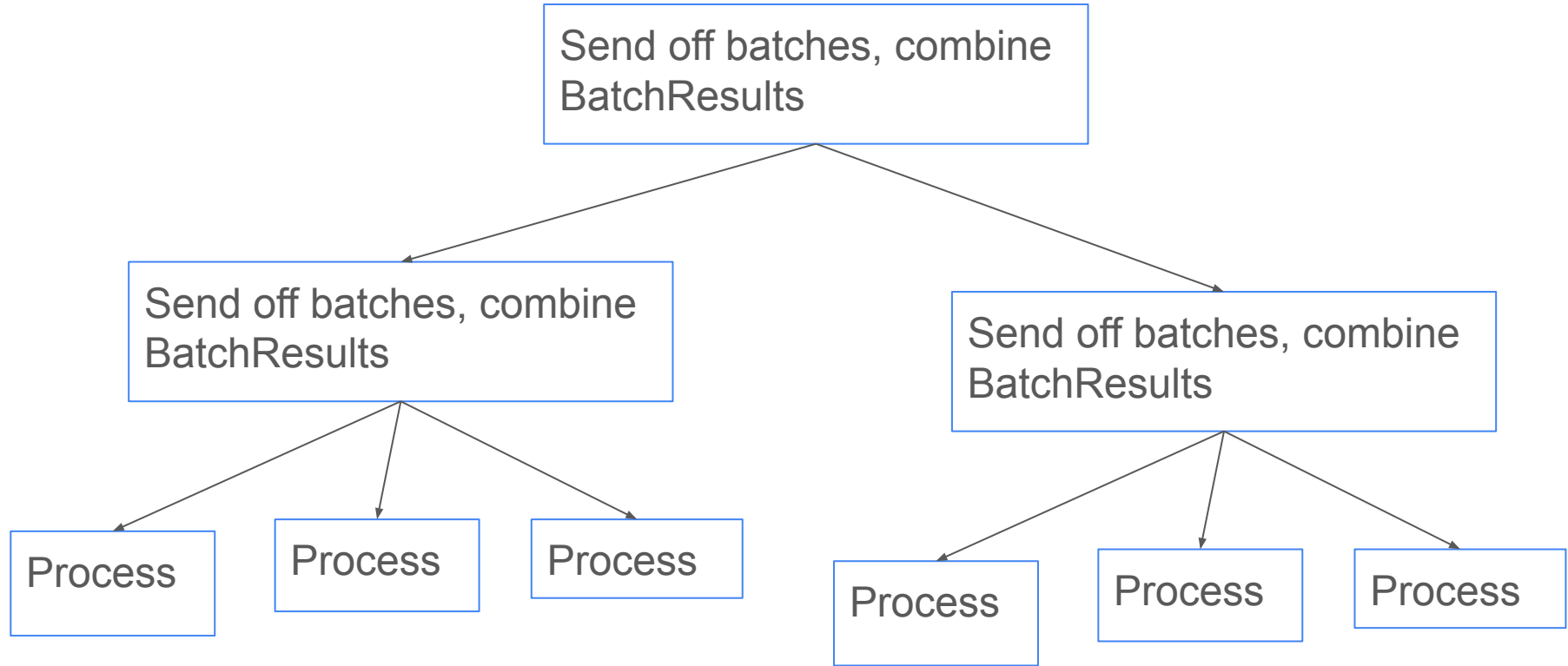
Step 1:

Create a method that will take in a `List<Integer>` and a number of shards, and create one batch of integers per shard

Combining Results



Combining Results



Your Turn!

Overall goal: build some multi-process 'find top 3 values' code

Step 2:

Create a method that will take in a batch of integers and return an initial "batch result" (what would be a convenient type?)

Step 3:

Create a method that will take in a list of "batch results" and return a combined "batch result"

MapReduce

"Map" - math term for 'transform'

"Reduce" - math term for 'combine into single value'

Process initial batch into BatchResult = map

Combine many BatchResults into one = reduce

MapReduce

Several frameworks for doing this IRL

MapReduce (Hadoop) and Spark are two of the most common (and closely related)

All the details:

MapReduce white paper:

<https://static.googleusercontent.com/media/research.google.com/en//archive/mapreduce-osdi04.pdf>

Spark white paper:

https://www.usenix.org/legacy/event/hotcloud10/tech/full_papers/Zaharia.pdf

Both integrate nicely with Java, either natively (Hadoop) or effectively so (Spark is written in Scala)

Fault tolerance - Hadoop

HDFS (HaDooP File System):

- Replicates data
- Every operation gets written to disk

⇒ Very fault tolerant

⇒ Low chance of having to redo computation

Downside: computations are usually cheaper than disk writes

Fault Tolerance - Spark

Avoids the overhead and io expense of Hadoop by keeping things in memory

But wait! What about failure recovery if everything is in **volatile** storage?

Solution:

RDD - Resilient Distributed Dataset

Rather than store the actual data, Spark (mostly) stores how the data was computed (lineage)

- Takes the lazy initialization pattern to another level

Spark - General Overview

Start with data either from code outside Spark (through a **driver** program) or loaded from a distributed file system (HDFS, s3, etc)

Execute MapReduce-like operations on it

- Adds an explicit "filter" operation to remove elements from a result set
- These map/reduce steps can be **chained** together to allow more complex logic
- In Spark land, each operation is called a "closure"

Collect (or save) results - this is what actually triggers any sort of computation

Spark

Generally preferred to Hadoop unless parallelism in the nested reduce steps is critical

- Multi-process means that RAM can be nearly unlimited, which adds a substantial speed-up over Hadoop (no extra i/o)
- Additional usability features (filters, accumulators, foreach) make it easier to adapt to program logic

Your Turn!

Create a main method for the 'find top 3' code. It should:

1. Load numbers from a file (format up to you: csv, 1 per line, space separated, etc)
2. Split the numbers into batches, and send them off to be run in parallel (for now, we'll use threads)
3. Add some additional batch-to-thread association, so that if a batch fails, it can be retried
4. Add a 10 second timeout to `future.get()`
5. Add retry logic in the catch block for `future.get()` for both exceptions and timeouts

What If It's Not Stateless

Sometimes, shared state is unavoidable

- locks
- caches
- file system data

⇒ Use a central server (single point of failure)

Multi-Process Computing

So....how to scale this central server?

Non-trivial, do not try to reinvent a solution yourself!

Options:

Databases - Vertica, Cassandra (Postgres HA is notoriously buggy, Oracle HA is notoriously \$\$\$\$)

Algorithms - Paxos

Enormous amounts of hardware everywhere - Spanner

Real Multi-Process Example

Problem: Incoming stream of BigInteger values, where one element occurs more than half the time. Find that element.

Ex:

1,1,2,2,1 $\Rightarrow$ 1

1,2,3 $\Rightarrow$ invalid input

2 $\Rightarrow$ 2

Real Multi-Process Example

Problem: Incoming stream of BigInteger values, where one element occurs more than half the time. Find that element.

Solution Constraints:

- Make sure each step of the process uses constant space (makes it easy to parallelize across machines)

SPOILERS AHEAD

Multi-Process Example

Problem: Incoming stream of BigInteger values, where one element occurs more than half the time. Find that element.

Key Insight (not obvious):

"more than half the time" = can pair instances of the element with non-instances of the element and still have some left over

Ex: 1,2,3,1,1

1 & 2, 1 & 3, and an extra 1 left over

Multi-Process Example

Problem: Incoming stream of BigInteger values, where one element occurs more than half the time. Find that element.

Track: best guess for element, number of "extra" of that element seen so far

When getting a new element from the stream:

- If it's the best guess, increment the number of "extra"; if 'best guess' is not set, set it to the current element and 'extra' to 1
- If it's not the best guess, decrement the number of "extra" by 1. If 'extra' is now 0, unset the best guess
- If the stream is done, return the best guess (or error if not set)

Let's write the code!

Multi-Process Example

How to parallelize?

Given an incoming stream, round-robin inputs to all the different servers

Take the results (plus the 'extra' counter) and consolidate as the final step
(central coordinating server)

- Note that now, the 'stream' length is bounded by the number of servers from part (1)

Multi-Process Example

Fun fact: If you change "more than $1/2$ " to "more than $1/k$ ", you can use this to create a multi-process streaming histogram algorithm that can quickly create pretty summaries of terabytes of data

Multi-Process Computing

Single servers have upper bounds on IO and CPU (fairly large these days, but still noticeable)

Threads require a shared heap, and therefore must run on one server

To continue to scale software, we need to move Beyond Threads

Splitting a job across multiple machines is called **sharding** (each machine is a **shard**)

Design For Multi-Process

Algorithm must be **shardable**

- Identify "batches" of work that can happen independently
- Send one batch to each shard

Overall result must be computable from results of each shard

Real Multi-Process Example

Problem: Incoming stream of BigInteger values, where one element occurs more than half the time. Find that element.

Ex:

1,1,2,2,1 $\Rightarrow$ 1

1,2,3 $\Rightarrow$ invalid input

2 $\Rightarrow$ 2

Real Multi-Process Example

Problem: Incoming stream of BigInteger values, where one element occurs more than half the time. Find that element.

Solution Constraints:

- Make sure each step of the process uses constant space (makes it easy to parallelize across machines)

SPOILERS AHEAD

Multi-Process Example

Problem: Incoming stream of BigInteger values, where one element occurs more than half the time. Find that element.

Key Insight (not obvious):

"more than half the time" = can pair instances of the element with non-instances of the element and still have some left over

Ex: 1,2,3,1,1

1 & 2, 1 & 3, and an extra 1 left over

Multi-Process Example

Problem: Incoming stream of BigInteger values, where one element occurs more than half the time. Find that element.

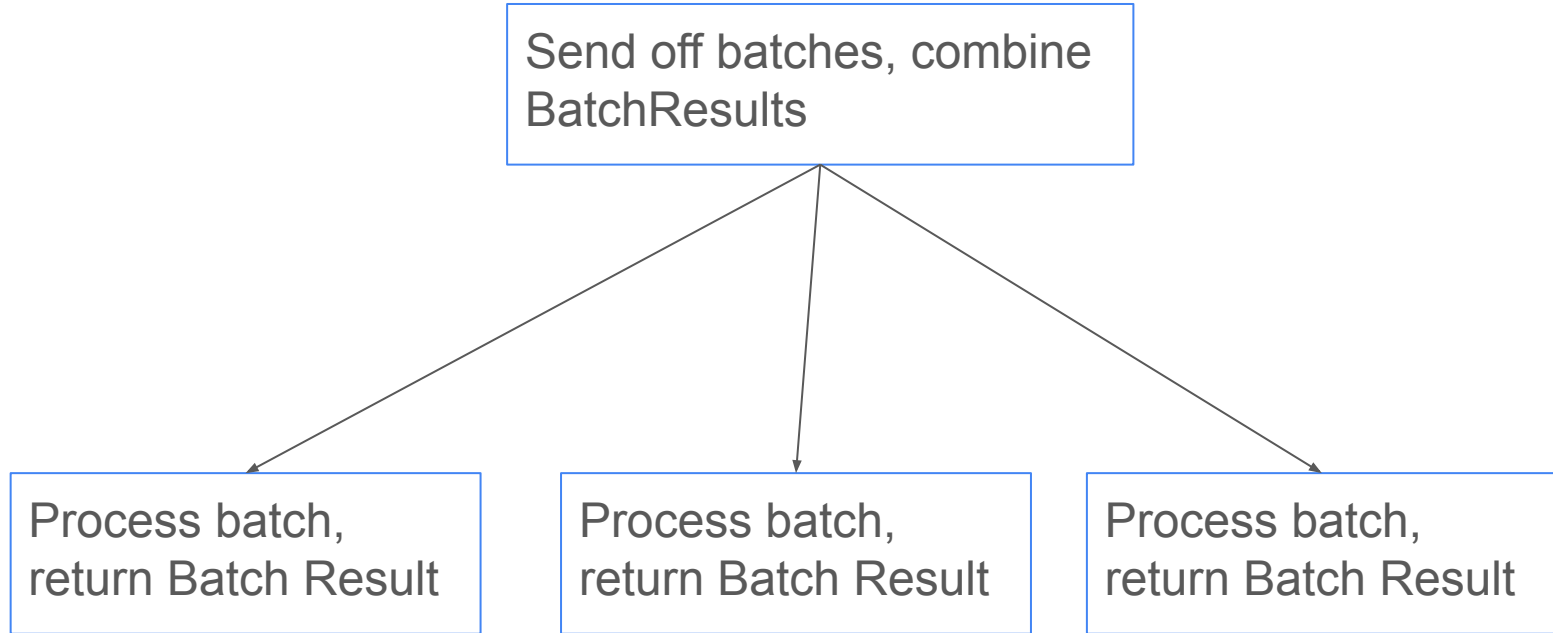
Track: best guess for element, number of "extra" of that element seen so far

When getting a new element from the stream:

- If it's the best guess, increment the number of "extra"; if 'best guess' is not set, set it to the current element and 'extra' to 1
- If it's not the best guess, decrement the number of "extra" by 1. If 'extra' is now 0, unset the best guess
- If the stream is done, return the best guess (or error if not set)

Let's write the code!

Combining Results



Modifying Result to Allow Re-combining

batches: {1,1,1,1,1}, {2,2,2,1,1}, {2,2,2,1,1}

Naive approach, get back:

{1}, {2}, {2}

Multi-Process Example

How to parallelize?

Given an incoming stream, round-robin inputs to all the different servers

Take the results (plus the 'extra' counter) and consolidate as the final step
(central coordinating server)

- Note that now, the 'stream' length is bounded by the number of servers from part (1)

What If the Data Arrived Earlier?

"Incoming stream from network" is a little weird

"Data built up over time in a program", on the other hand...

Pre-Stored Batches

Have each shard have data + process

- Batch is whatever data is stored locally

How to Make This Useful

IRL, this went the other way:

- simplify a complex problem down to a small situation
- solve the simple case
- regeneralize the solution

How to Make This Useful

Original Problem:

10TB* of structured data

User wants a histogram of some feature (ex: country codes for phone numbers),
as quickly as possible (<10 seconds)

- Features are unknown ahead of time (user-defined schema)

How to Make This Useful

Digging into requirements:

- User doesn't care about the long tail, they want the top "several"
- User would prefer the program decide how many is "several" depending on the data

How to Make This Useful

Simple case:

- One item occurs more than $1/2$ the time; return that item and an 'other' bucket

How to Make This Useful

More complex case:

- One item occurs more than $1/k$ of the time: same algorithm, but storing (up to) $k-1$ items

How to Make This Useful

Putting it all together:

- Pick a plausibly large value of k that can also easily fit in memory (100 ish, configurable)
- Return counts for everything that shows up more than 1% of the time
- If nothing returned (unusual): add an order of magnitude to k , try again

How to Make This Useful

Still needs to be fast:

10 TB of RAM is hard to get on one machine

784 GB is available on EC2, though

⇒ a few dozen servers and the whole thing is cached in memory